



WAZUH SIEM DEPLOYMENT AND CONFIGURATION

Setting Up Centralized Monitoring with Wazuh

Configuring File Integrity Monitoring and Confirming Alerts Work End to End

Prepared By	Abdullah Masood
Program	Blue Team Internship
Organization	ITSimplera Institute
Reviewed By	Instructor Zia Ullah
Date	4 July 2026

Contents

Contents	2
Project Overview	3
What I Set Out to Do	3
What the Task Asked For	3
1. Getting Wazuh Running	4
2. Building the Windows and Kali Machines	Error! Bookmark not defined.
Kali Linux (Endpoint).....	6
Windows Endpoint	Error! Bookmark not defined.
3. Bringing Up the Wazuh Dashboard.....	6
4. Basic Configuration	Error! Bookmark not defined.
5. Connecting the Windows Agent	Error! Bookmark not defined.
6. Connecting the Kali Linux Agent	Error! Bookmark not defined.
7. Setting Up and Testing File Integrity Monitoring	Error! Bookmark not defined.
8. Reviewing the Alerts	Error! Bookmark not defined.
9. Wrapping Up	15

Project Overview

Deploying a Small Blue Team Lab: Wazuh Manager with Windows and Linux Endpoints

1. Objective

The objective of this lab was to deploy the Wazuh security platform, connect Windows and Kali Linux agents, configure File Integrity Monitoring (FIM), and verify that Wazuh detects file changes in real time.

2. Tools Used

- Ubuntu Server (Wazuh Manager)
- Windows 11
- Kali Linux
- VMware Workstation
- Wazuh Dashboard
- PowerShell
- Terminal (Linux)

What I Set Out to Do

Coming in as a Blue Team intern at ITSimplera Institute, my first assignment was to stand up a working Wazuh SIEM lab and use it to keep an eye on more than one machine at once. I hooked up a Windows box and a Kali Linux box as agents, turned on File Integrity Monitoring, and then double-checked that the alerts it generates actually show up correctly. Beyond the technical steps, the real value of this exercise was getting a feel for how a Security Operations Center actually functions day to day, which sets me up for the more advanced Blue Team work ahead.

What the Task Asked For

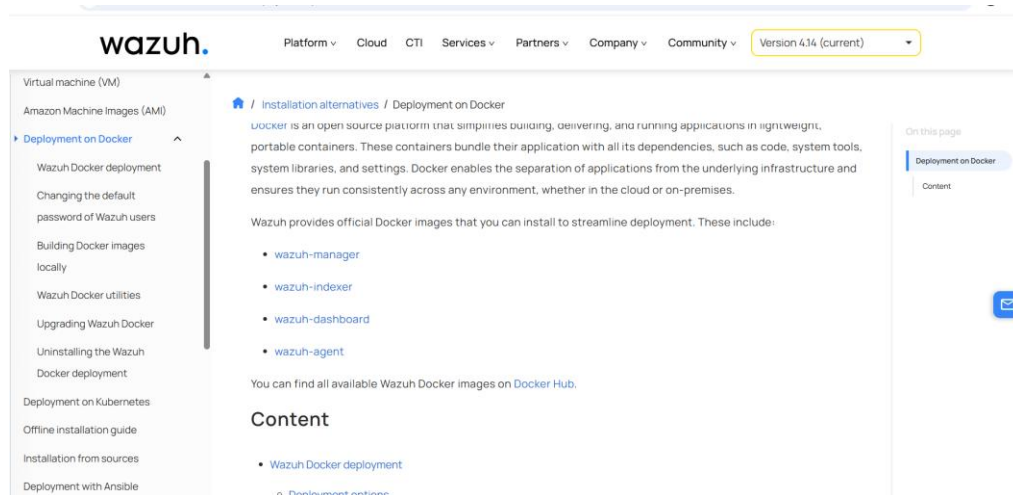
- Get the Wazuh virtual appliance up and running
- Configure the Wazuh Manager
- Install agents on both a Windows machine and a Kali Linux machine
- Enroll each agent with the manager
- Turn on File Integrity Monitoring and confirm it actually works
- Trigger some test activity and check that alerts are generated
- Write down what I did at every stage

1. Getting Wazuh Running

There's more than one way to stand up Wazuh, but I went with the route that involves the least manual assembly: the ready-made virtual machine image Wazuh distributes as an OVA file. That single image bundles the manager, the indexer, and the dashboard together, so I didn't have to install or wire up each piece separately.

I grabbed the OVA file straight from Wazuh's own documentation site and brought it into VMWare. That same page also spells out the minimum hardware needed and which package version pairs with it — I ended up on 4.14.6.

Once the appliance had booted, I went into its network settings so it could talk to the outside world as well as the other VMs in the lab so I left the main adapter on NAT for internet access.



[/ Installation alternatives](#) / Virtual machine (VM)

VM runs only on 64-bit systems with x86_64/AMD64 architecture. It does not provide high availability or scalability out of the box. However, you can implement these using [distributed deployment](#).

Download the [virtual appliance \(OVA\)](#).

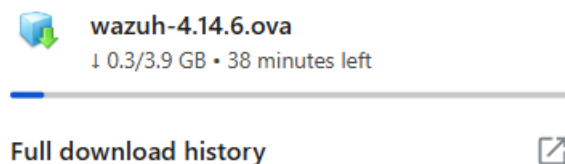
OS	Architecture	VM Format	Version	Package
Amazon Linux 2023	64-bit x86_64/AMD64 architecture	OVA	4.14.6	wazuh-4.14.6.ova (sha512)

Hardware requirements

The following requirements have to be in place before the Wazuh VM can be imported into a host operating system:

- The host operating system must be 64-bit with x86_64/AMD64 architecture.

Here clicking blue line in package column then starts downloading Ova file.



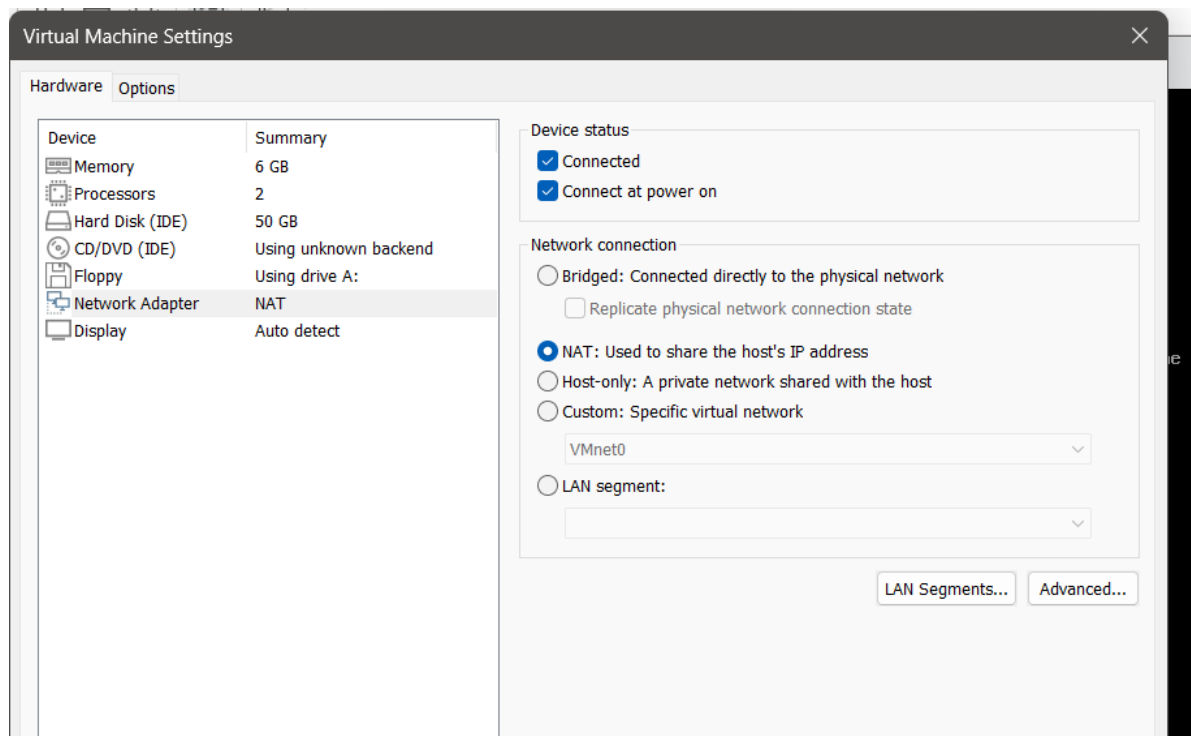
Here Ova file is downloading

2. Building the Windows and Kali Machines

Two more virtual machines went into this lab: a Windows 11 pro box to stand in for a normal office workstation, and a Kali Linux box to represent a Linux endpoint. Both were built the standard way, using their official installer images inside VMware. Both act as Agent names Windows and Linux Agent.

Windows VM (Wazuh VM)

Windows is what most real organizations run on their desktops, so having it in the lab means testing the monitoring setup against something closer to a real workstation. I used the standard guided installer, created a local account, and once I reached the desktop I checked the network adapters the same way I did for Kali — one for internal lab communication, one for internet.



Create a new virtual machine named WazuhVM where importing ova file to Vmx file then going to VM Settings where Changing Network Adapter to NAT for internet access and I suit memory according to my laptop RAM usable storage like I do 6GB and select no of processor a 1 where each number of processor has two.

Kali Linux (Endpoint)

Kali plays the role of the Linux endpoint here. I have already Kali Linux where I used it as Linux Agent to Wazuh Server.

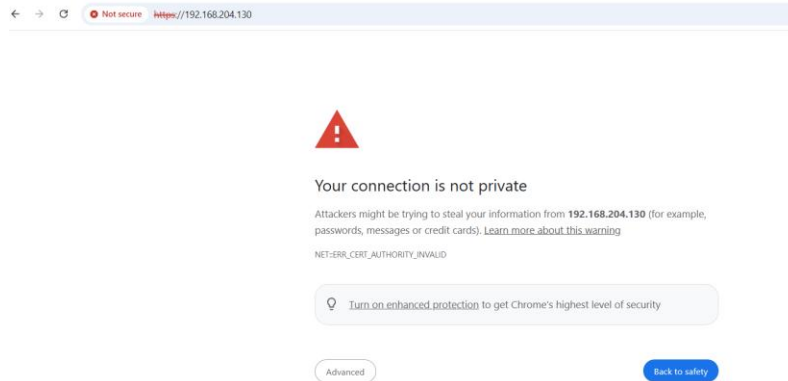
3. Bringing Up the Wazuh Dashboard

```

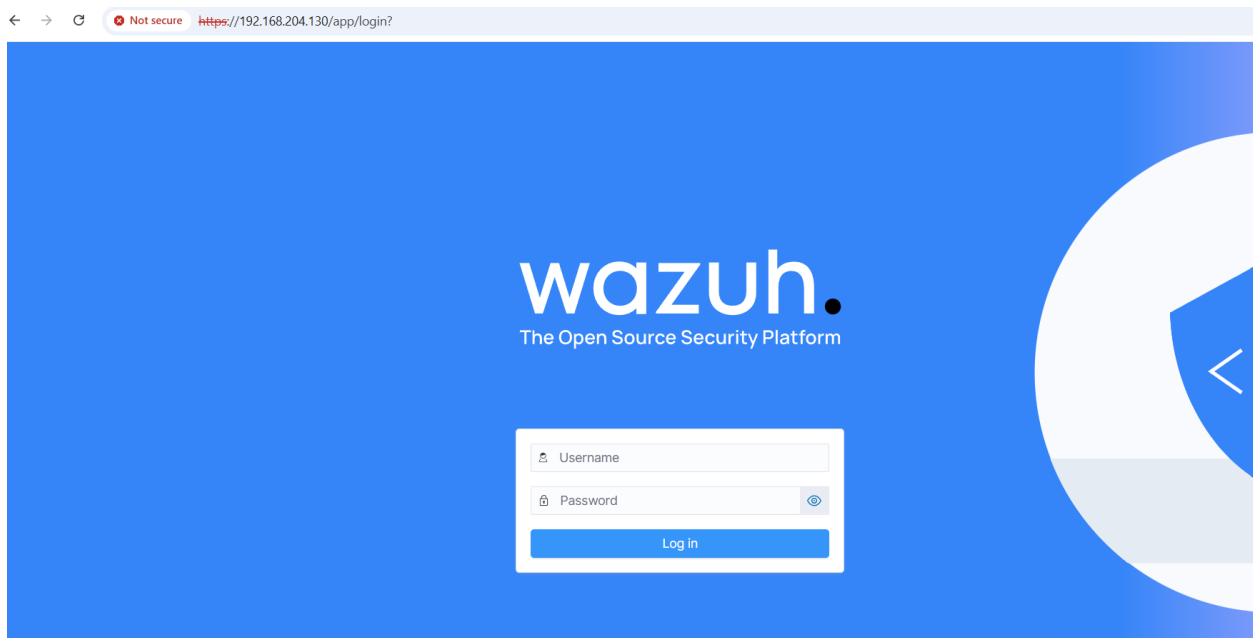
https://wazuh.com
[wazuh-user@wazuh-server ~]# Sip a
-bash: Sip: command not found
[wazuh-user@wazuh-server ~]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e3:f7:94 brd ff:ff:ff:ff:ff:ff
    altname enp2s0
    altname ens32
    inet 192.168.204.130/24 metric 1024 brd 192.168.204.255 scope global dynamic eth0
        valid_lft 1523sec preferred_lft 1523sec
    inet6 fe80::20c:29ff:fee3:f794/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
[wazuh-user@wazuh-server ~]#

```

With the appliance running, I pointed a browser at WazuhVM IP address to reach the Wazuh dashboard's login screen, and signed in with the default administrator account that gets created during setup.

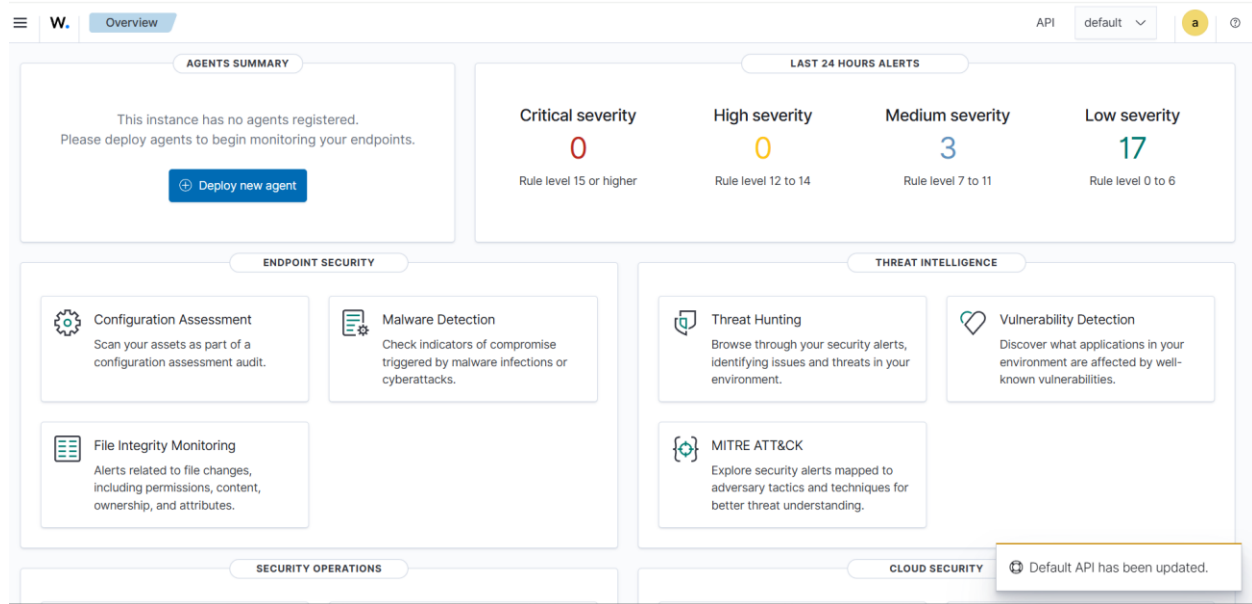


Because webpage was not secure then I go to advanced to open the page.



Signing in to the Wazuh dashboard for the first time.

The overview screen comes up right after login. Since no agents had been added yet, the agent summary was naturally empty — nothing unusual, just a sign that the endpoints hadn't been connected.



4. Basic Configuration

Out of the box, the Wazuh manager's default settings are sensible enough that I didn't need to touch much on the server side for a lab of this size. The overview screen already lists the modules running by default — File Integrity Monitoring, Malware Detection, Threat Hunting, and Vulnerability Detection — and those are the ones this report focuses on.

The real configuration work happened on the agent side: every endpoint has to be told which IP address to report back to. That gets set at install time, and I cover exactly how in the two deployment sections below.

5. Installing the Windows Agent

Step 1

Opened **Deploy New Agent** from the Wazuh Dashboard.

Step 2

Selected

- Windows
- MSI 64-bit

Entered

- Server IP: **192.168.204.130**

- Agent Name: **AbdullahPC**

<

Deploy new agent

LINUX

☐ RPM amd64

☐ RPM aarch64

☐ DEB amd64

☐ DEB aarch64

WINDOWS

☒ MSI 32/64 bits

macOS

☐ Intel

☐ Apple silicon

For additional systems and architectures, please check our [documentation](#).

✓

Server address:

This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FQDN).

Assign a server address

192.168.204.130

☐ Remember server address

3

Optional settings:

By default, the deployment uses the hostname as the agent name. Optionally, you can use a different agent name in the field below.

Assign an agent name

Windows-PC

The character "-" is not valid. Allowed characters are A-Z, a-z, 0-9, ".", "_", "-", "~", ":", ";", "!", "@", "#", "\$", "%", "&".

Select one or more existing groups:

default

4

Run the following commands to download and install the agent:

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.6-1.msi -OutFile $env:tmp\wazuh-agent; msixec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER="192.168.204.130" WAZUH_AGENT_GROUP="default" WAZUH_AGENT_NAME="AbdullahPC"
```

Requirements

- You will need administrator privileges to perform this installation.
- PowerShell 3.0 or greater is required.

Keep in mind you need to run this command in a Windows PowerShell terminal.

5

Start the agent:

```
NET START Wazuh
```

6

Go to endpoints to verify the agent connection:

Back to agent list

Step 3

Executed the generated PowerShell command as Administrator.

Step 4

Started the agent using

```
NET START Wazuh
```

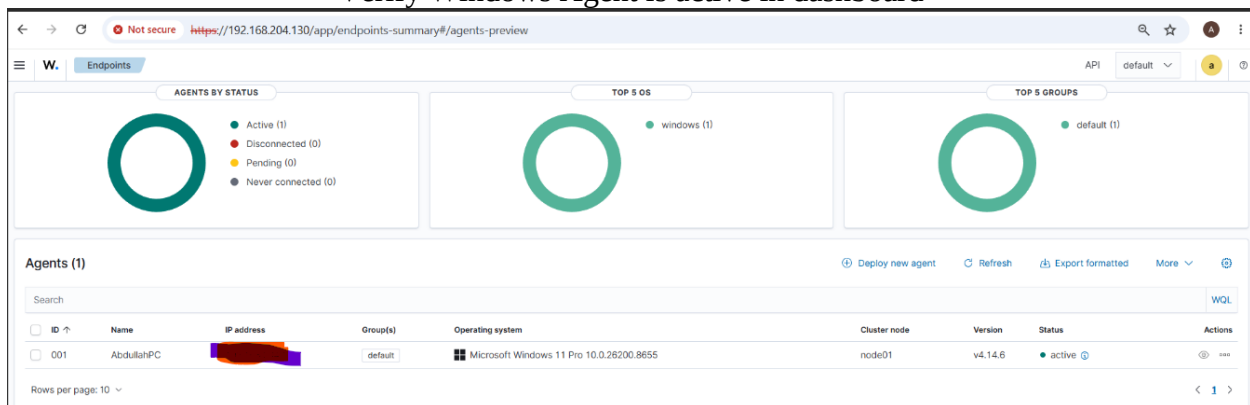
The service started successfully.

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.6-1.msi -OutFile
$env:tmp\wazuh-agent; msixec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER='192.168.204.130' WAZUH_AGENT_GROUP='default'
WAZUH_AGENT_NAME='AbdullahPC'
PS C:\WINDOWS\system32> NET START Wazuh
The Wazuh service is starting.
The Wazuh service was started successfully.
```

Verify Windows Agent is active in dashboard



6. Installing the Kali Linux Agent

Selected

- Linux
- DEB amd64

Select the package to download and install on your system:

LINUX

☐ RPM amd64 ☐ RPM aarch64

☒ DEB amd64 ☐ DEB aarch64

WINDOWS

☐ MSI 32/64 bits

macOS

☐ Intel ☐ Apple silicon

For additional systems and architectures, please check our [documentation](#).

Server address:

This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FQDN).

Assign a server address

192.168.204.130

☒ Remember server address

Optional settings:

By default, the deployment uses the hostname as the agent name. Optionally, you can use a different agent name in the field below.

Assign an agent name

Kali

The agent name must be unique. It can't be changed once the agent has been enrolled.

Select one or more existing groups: ⓘ

Default ▼

4 Run the following commands to download and install the agent:

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.6-1_amd64.deb && sudo WAZUH_MANAGER='192.168.204.130' WAZUH_AGENT_NAME='Kali' dpkg -i ./wazuh-agent_4.14.6-1_amd64.deb
```

ⓘ Requirements

- You will need administrator privileges to perform this installation.
- Shell Bash is required.

Keep in mind you need to run this command in a Shell Bash terminal.

5 Start the agent:

```
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

6 Go to endpoints to verify the agent connection:

[Back to agent list](#)

Used the installation command generated by Wazuh.

Started the agent using

```
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

```
(kali@kali) [/home/kali]
PS> wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.6-1_amd64.deb 66 sudo WAZUH_MANAGER='192.168.204.130' WAZUH_AGENT_NAME='Kali' dpkg -i ./wazuh-agent_4.14.6-1_amd64.deb
--2026-07-04 05:48:09-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.6-1_amd64.deb
Resolving packages.wazuh.com (packages.wazuh.com) ... 18.64.141.3, 18.64.141.53, 18.64.141.68, ...
Connecting to packages.wazuh.com (packages.wazuh.com)|18.64.141.3|:443 ... connected.
HTTP request sent, awaiting response ... 200 OK
Length: 13220582 (13M) [application/vnd.debian.binary-package]
Saving to: 'wazuh-agent_4.14.6-1_amd64.deb'

wazuh-agent_4.14.6-1_amd64.deb 100%[=====] 12.61M 2.03MB/s in 7.6s

2026-07-04 05:48:17 (1.66 MB/s) - 'wazuh-agent_4.14.6-1_amd64.deb' saved [13220582/13220582]

[sudo] password for kali:
Selecting previously unselected package wazuh-agent.
(Reading database ... 450439 files and directories currently installed.)
Preparing to unpack ./wazuh-agent_4.14.6-1_amd64.deb...
Unpacking wazuh-agent (4.14.6-1)...
Setting up wazuh-agent (4.14.6-1)...
```

```
(kali@kali) [/home/kali]
PS> sudo systemctl daemon-reload
(kali@kali) [/home/kali]
PS> sudo systemctl enable wazuh-agent
Created symlink '/etc/systemd/system/multi-user.target.wants/wazuh-agent.service' → '/usr/lib/systemd/system/wazuh-agent.service'.
(kali@kali) [/home/kali]
PS> sudo systemctl start wazuh-agent
```

The install command and start-service instructions generated by the wizard.

Agent Verification

Agents (2) Deploy new agent Refresh Export formatted More WQL

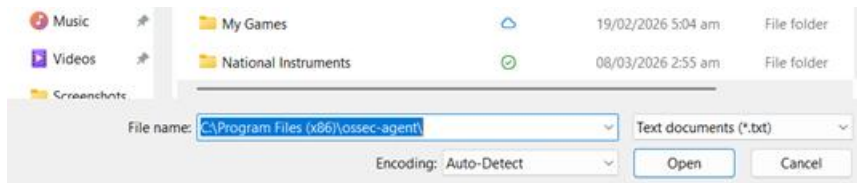
status=active

ID ↑	Name	IP address	Group(s)	Operating system	Cluster node	Version	Status	Actions
001	AbdullahPC	[REDACTED]	default	Microsoft Windows 11 Pro 10.0.26200.8655	node01	v4.14.6	active	...
002	Kali	[REDACTED]	default	Kali GNU/Linux 2025.4	node01	v4.14.6	active	...

Both agents work under same Manager and Server of Wazuh.

7. Configuring File Integrity Monitoring

File Integrity Monitoring watches for changes to files and folders on an endpoint — creation, edits, deletions, all of it. To turn it on for the Kali agent, I opened its ossec.conf configuration file with root access.



Edited

C:\Program Files (x86)\ossec-agent\ossec.conf

Added the Desktop folder for monitoring in ossec.conf

```
<!-- File integrity monitoring -->
<syscheck>
<disabled>no</disabled>

<frequency>43200</frequency>

<directories realtime="yes" check_all="yes">C:\Users\Abdullah\OneDrive\Desktop</directories>
```

Edit the ossec.conf file which is in desktop folder of program files x86.

```
PS C:\WINDOWS\system32> Get-Content "C:\Program Files (x86)\ossec-agent\ossec.conf"
```

Putting this command in powershell displays all text of ossec.log file

Restarted the Wazuh Agent.

```
PS C:\WINDOWS\system32> NET STOP Wazuh

The Wazuh service was stopped successfully.

PS C:\WINDOWS\system32> NET START Wazuh

The Wazuh service was started successfully.
```

Verify the ossec.log are also working by opening it

```
2026/07/04 15:33:50 wazuh-agent: INFO: (6207): Ignore 'registry' sregex '\Enum$'
2026/07/04 15:33:50 wazuh-agent: INFO: Started (pid: 1120).
2026/07/04 15:33:50 sca: INFO: Module started.
2026/07/04 15:33:50 wazuh-modulesd:ciscat: INFO: Module disabled. Exiting...
2026/07/04 15:33:50 sca: INFO: Loaded policy 'C:\Program Files (x86)\ossec-agent\ruleset\sca\cis_win11_enterprise.yml'
2026/07/04 15:33:50 sca: INFO: Starting Security Configuration Assessment scan.
2026/07/04 15:33:50 wazuh-agent: INFO: Using AES as encryption method.
2026/07/04 15:33:50 wazuh-agent: INFO: Trying to connect to server ([192.168.204.130]:1514/tcp).
2026/07/04 15:33:50 wazuh-modulesd:osquery: INFO: Module disabled. Exiting...
2026/07/04 15:33:50 wazuh-modulesd:syscollector: INFO: Module started.
2026/07/04 15:33:50 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/07/04 15:33:50 sca: INFO: Starting evaluation of policy: 'C:\Program Files (x86)\ossec-agent\ruleset\sca\cis_win11_enterprise.yml'
2026/07/04 15:33:50 wazuh-agent: INFO: (6000): Starting daemon...
2026/07/04 15:33:50 wazuh-agent: INFO: (6010): File integrity monitoring scan frequency: 43200 seconds
2026/07/04 15:33:50 wazuh-agent: INFO: (6008): File integrity monitoring scan started.
2026/07/04 15:33:50 wazuh-agent: INFO: (4102): Connected to the server ([192.168.204.130]:1514/tcp).
2026/07/04 15:33:50 rootcheck: INFO: Starting rootcheck scan.
2026/07/04 15:33:50 wazuh-agent: INFO: Started (pid: 1120).
2026/07/04 15:33:52 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/07/04 15:33:55 wazuh-agent: INFO: Agent is now online. Process unlocked, continuing...
2026/07/04 15:33:55 rootcheck: INFO: Ending rootcheck scan.
2026/07/04 15:34:03 sca: INFO: Evaluation finished for policy 'C:\Program Files (x86)\ossec-agent\ruleset\sca\cis_win11_enterprise.yml'
2026/07/04 15:34:03 sca: INFO: Security Configuration Assessment scan finished. Duration: 13 seconds.
2026/07/04 15:34:08 wazuh-agent: INFO: (6009): File integrity monitoring scan ended.
2026/07/04 15:34:08 wazuh-agent: INFO: FIM sync module started.
2026/07/04 15:34:08 wazuh-agent: INFO: (6012): Real-time file integrity monitoring started.
```

8. Testing File Integrity Monitoring

To actually put FIM to the test, I created a new file, put some text in it, and then deleted it — basically mimicking the kind of file activity FIM is supposed to catch on a real machine. I create file named new.txt then rename it to wazuh-test.txt and add some text inside the file. I delete the existence file named newabc.txt

Created

new.txt

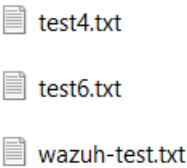
Modified

wazuh-test.txt

Deleted

newabc.txt

Wazuh generated alerts immediately.



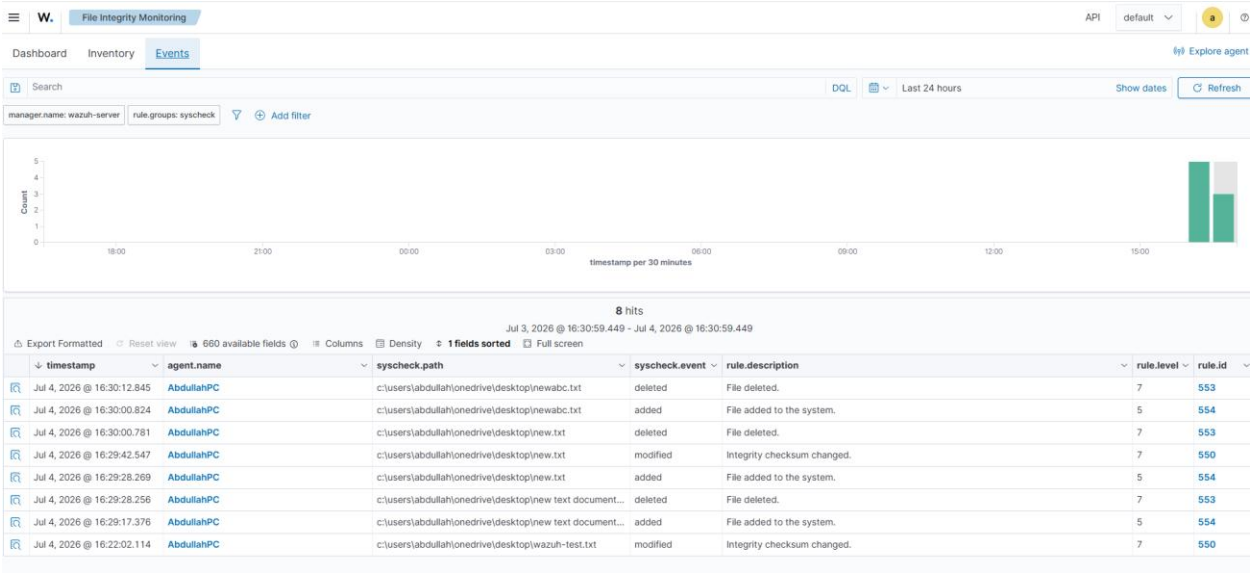
9. Results

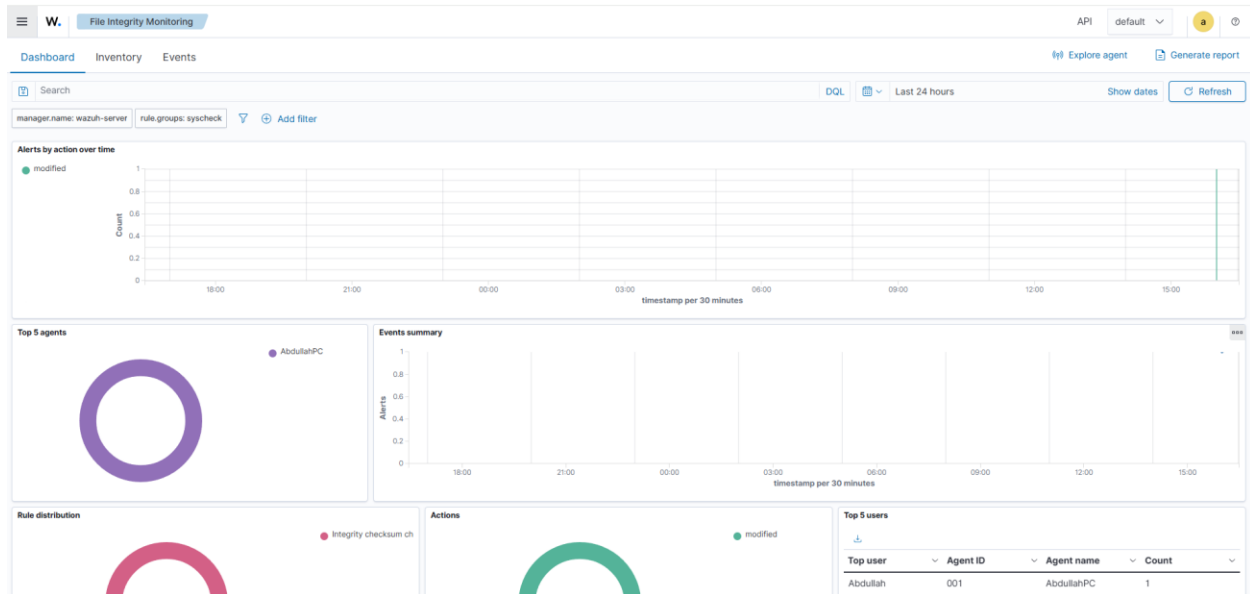
The Wazuh dashboard detected

- File Added
- File Modified
- File Deleted

Examples

File	Event
wazuh-test.txt	Modified
new.txt	Added
new.txt	Modified
new.txt	Deleted
newabc.txt	Added
newabc.txt	Deleted





10. Conclusion

The Wazuh platform was successfully deployed and configured. Windows and Kali Linux agents communicated successfully with the Wazuh Manager. File Integrity Monitoring correctly detected file creation, modification, and deletion events in real time. The alerts appeared in the Wazuh Dashboard, demonstrating that Wazuh can effectively monitor file system changes and improve endpoint security.

This assignment took me through a compact but complete SOC setup from start to finish — standing up the Wazuh manager, connecting two very different endpoints to it, and proving that File Integrity Monitoring genuinely produces alerts you can see and dig into on the dashboard. More than anything, it gave me a practical feel for how centralized security monitoring actually works and it's a solid launchpad for the Blue Team tasks that come next.